ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о командном программном проекте на тему:

Разработка прототипа симулятора для тактических игр на примере Zebra Puzzle

(промежуточный, этап 1)

Выполнен студентами:

группы БПМИ 237, 3 курса	Шарипов Нуриддин Заурович
группы БПМИ 238, 3 курса	Гоибов Дододжон Абдуназарович
группы БПМИ 2310, 3 курса	Темуров Мухаммадали Абдужабборович

Принял руководитель проекта:

Хельвас Александр Валериевич
Генеральный директор
ООО “СОККОС”

Москва 2026

Руководители

Александр Валериевич Хельвас , ктн

Khelvas Aleksandr Valerievich

Студенты на программном проекте

Шарипов Нуриддин Заурович

Sharipov Nuriddin Zaurovich

nzsharipov@edu.hse.ru

Гоибов Дододжон Абдуназарович

Ghoibov Dadojon Abdunazarovich

dagoibov@edu.hse.ru

Темуров Мухаммадали Абдужабборович

Temurov Muhammadali Abduzhabborovich

matemurov@edu.hse.ru

Реферат

СИТУАЦИОННАЯ ОСВЕДОМЛЕННОСТЬ,ЗАДАЧА ЭЙНШТЕЙНА, ZEBRA PUZZLE, ТАКТИЧЕСКАЯ ИГРА, TELEGRAM BOT, REST API, ДИСКРЕТНОЕ СОБЫТИЙНОЕ МОДЕЛИРОВАНИЕ, ЛОГИРОВАНИЕ, CSV, XML, ГРАФИК SA(t), МЕТРИКИ M1/M2.

Ситуационная осведомлённость (СО, англ. Situation Awareness, SA) - это способность субъекта (человека, системы или организации) воспринимать, анализировать и прогнозировать состояние окружающей среды и собственного положения в динамически изменяющихся условиях. В работе разработана программная часть прототипа тактической игры на основе сюжета Zebra Puzzle.

Прототип реализован в виде клиент-серверной системы. Серверная часть предоставляет REST API и обеспечивает создание и ведение игровой сессии, хранение состояния, прием действий участников, пошаговый запуск дискретно-событийной симуляции и завершение моделирования. Клиентская часть реализована в виде Telegram Bot и обеспечивает игровой интерфейс: создание лобби, подключение участников, запуск сессии, выдачу игроку актуального состояния на каждом ходе (позиция, доступные действия, известные факты и неопределенности), а также уведомления о событиях, влияющих на SA (перемещения, посещения, встречи, взаимодействия). Поддерживается смешанный состав участников (реальные игроки и компьютерные агенты); игроки, не присоединившиеся в течение времени TimeDelay, автоматически заменяются симуляторами.

Результаты моделирования протоколируются с формированием лог-файлов в форматах CSV и XML, содержащих состояние и события по шагам, а также значения метрик SA. На основе логов подготавливаются данные для построения графиков зависимости SA(t) для каждого участника и сравнения стратегий поведения.

Содержание

1 Постановка задачи ситуационной осведомленности 8
1.1 Формализованная постановка задачи 8
1.2 Метрики ситуационной осведомленности для дискретных событийно управляемых систем 10
2 Постановка задачи на примере Zebra Puzzle 12
2.1 Постановка задачи оптимальной стратегии агентов . . . 18
3 Постановка задачи на разработку тактической игры 19
3.1 Разработка базовой технологической платформы 19
3.2 Разработка решения для симуляции задачи 20
3.3 Разработка программной реализации алгоритма действия игрока 21
3.4 Разработка решения для анализа результатов моделирования 21
3.5 Архитектура и компоненты системы 22
3.6 Реализованный функционал на текущий момент 22
4 Обзор литературы 24
4.1 История и современные представления о SA 24
4.2 Использование ZEBRA PUZZLE для модельных задач . 25
4.3 Варианты решений для решателей включая FOL 25
4.4 Теоретико-игровой подход к задаче SA 26
4.5 Обзор игровых симуляторов 26
4.6 Обзор DES решений для задач SA 28
5 Описание решения 30
5.1 Простая модель поведения 30
5.2 Метрики ситуационной осведомленности 30
5.3 Метрики ситуационной осведомленности в простейшем случае 31
5.4 Метрика "У кого зебра" 31
6 Результаты 32
6.1 Описание архитектуры игрового сервиса 32
6.1.1 Состав компонент 32
6.1.2 Интерфейсы и обмен данными 33

6.1.3 Жизненный цикл игры 34
6.1.4 Механика знаний и событий, влияющих на SA . . 35
6.1.5 Логирование и подготовка данных для анализа . 35
Заключение 36
Список использованных источников 38
А Пример диалогов в Telegram 41

Обозначения

$A = \{A_1, A_2. \ldots, A_n\}$ – множество подсистем входящих в систему;

$B = \{B_1, B_2. \ldots, B_i\}$ – множество объектов, входящих в систему;

$E = \{e_1, e_2, \ldots, e_m\}$ – множество событий в системе;

$Q \subseteq \mathbb{E} \times \mathbb{T} \times \mathbb{P}$ — очередь событий (упорядоченное множество троек "событие-время-параметры") в системе;

Q_P – упорядоченный список уже произошедших событий;

Q_F – упорядоченный список событий, которые еще не произошли;

$\mathbb{S} : \mathbb{B} \times \mathbb{T} \to \mathbb{S}$ — функция состояния объекта в момент времени;

$S^n = \{s_1^n, s_2^n, \ldots, s_m^n\}$ – множество состояний n -го объекта в системе;

$\Sigma_t = \{s_t^1 . s_t^2, \ldots, s_t^I\}$ — состояние системы в момент времени t

ΔT – минимальный шаг моделирования по времени;

$\text{last_update} : \mathbb{B} \to \mathbb{T}$ — время последнего обновления объекта

$\text{current_time} \in \mathbb{T}$ — текущее системное время

Введение

Ситуационная осведомлённость (СО, англ. Situation Awareness, SA) —- это способность субъекта (человека, системы или организации) воспринимать, анализировать и прогнозировать состояние окружающей среды и собственного положения в динамически изменяющихся условиях.

Модель СО, предложенная М. Эндсли (Endsley, 1995) [1, 2], включает три уровня:

1) Восприятие (Perception), под которым обычно понимают сбор и обработку данных о текущем состоянии системы.

2) Осмысление (Comprehension) — интерпретация метрик системы, оценка их значимости.

3) Прогнозирование (Projection) — предсказание будущих состояний системы на основе текущего состояния и истории динамики системы.

Задача СО может быть представлена как задача оптимизации, в рамках которой необходимо:

- Максимизировать точность и полноту представления о ситуации.
- Минимизировать задержки в обработке информации
- Минимизировать количество ошибок.

Разделение ответственности в команде

Архитектурное разделение напрямую соответствует распределению задач:

— **НШ** - технологическая платформа: Telegram Bot, интеграция с REST API, управление сессией, UX игрового процесса, выгрузка логов.

— **ДГ** - симуляционное ядро и автономная симуляция: правила мира, события, корректность переходов состояния, полнота логирования.

— **МТ** - стратегия действий игрока: алгоритм выбора решений, взаимодействие с API, эксперименты и сравнение по SA(t).

1 Постановка задачи ситуационной осведомленности

1.1 Формализованная постановка задачи

Сложная дискретная система — это система, состоящая из набора составных частей – подсистем $A = \{A_1, A_2. \ldots, A_n\}$, , образующих иерархию. Состояние такой системы изменяется в дискретные моменты времени T_m в результате наступления событий e_m, локализованных во времени.

Объекты $B = \{B_1, B_2. \ldots, B_i\}$ — это независимые сущности системы не имеющие внутренней структуры и описываемые некоторым состоянием.

Каждому объекту B_n ставится в соответствие набор состояний $S^n = \{s_1^n, s_2^n, \ldots, s_{M^n}^n\}$. Состояние s_m^n изменяется исключительно под влиянием некоторого события. Каждый объект имеет собственное локальное время и состояние, которое может быть обновлено.

События $Q = \{q_1, q_2, \ldots, q_m\}$ — это записи, указывающие на то, что в определенный момент модельного времени с одним или несколькими объектами произошло или должно произойти некоторое изменение.

Типы событий будем обозначать $E = \{e_1, e_2, \ldots, e_m\}$.

Упорядочивая множество событий мы получаем очередь событий (упорядоченное множество троек "событие-время-параметры") $Q \subseteq \mathbb{Q} \times \mathbb{T} \times \mathbb{P}$. Очередь разбивается на Q_P – упорядоченный список уже произошедших событий и Q_F – упорядоченный список событий, которые еще не произошли (очередь ближайших событий).

Для каждого объекта вводим функцию состояния объекта в момент времени:

$$\mathbb{S}(b,t) : \mathbb{B} \times \mathbb{T} \to \mathbb{S}. \tag{1.1}$$

Инструментом моделирования таких систем является Дискретно-событийное моделирование (Discrete-Event Simulation — DES), основные подходы которого были сформулированы более 50 лет назад. Классический алгоритм DES в своей основе имеет список будущих событий (Future

Event Set, FES), из которого извлекается ближайшее событие. Это событие изменяет состояние одного или нескольких объектов и возвращает список новых событий, которые добавляются в FES. Далее цикл повторяется.

Будем предполагать что рядом с системой или ее моделью существует один или несколько наблюдателей (observer) $\mathbb{O} = \{O_1, O_2, \ldots, O_i\}$, которые ведут наблюдение за системой или моделью системы. При этом каждому наблюдателю O_i ставится в соответствие некоторая функция, которая может возвращать параметры некоторого события:

$$M^q{}_i(q_m) \to \hat{q}^i_m \tag{1.2}$$

.

Также наблюдателю O_i может ставится в соответствие некоторая функция, возвращающая полученное наблюдателем состояние объекта или нескольких объектов:

$$M^s{}_i(B_m) \to \widehat{s.B^i_m} \tag{1.3}$$

.

Кроме обычного наблюдателя можно ввести понятие активного наблюдателя (active listener), который кроме получения параметров некоторого события, может также планировать новые события в системе и являться частью системы. Так, например, при выполнении моделирования боевых действий, БПЛА разведывательного типа одновременно является и объектом системы и наблюдателем, который может получать информацию о некоторых событиях системы. Анализ оптимальных стратегий активных наблюдателей не входит в рамки данной статьи. Планирование события и последующее наблюдение за ним в паре образуют измерение.

1.2 Метрики ситуационной осведомленности для дискретных событийно управляемых систем

В рамках предложенного подхода мы можем для каждого момента времени T_k сформулировать набор фактов $\mathcal{F}^k = \{f_1^k, \ldots, f_W^k\}$, полностью описывающих систему для внешнего наблюдателя.

Всякий раз, когда говорят о ситуационной осведомленности, как правило под эти понимают осведомленность некоторого субъекта - наблюдателя. Мы считаем, что система описывается некоторым изменяющимся во времени набором фактов о состоянии объектов и о событиях, которые порождают изменения состояний объектов.

Будем понимать под метрикой ситуационной осведомленности наблюдателя L_n некоторую функцию $M_n^k = M(F_n^k)$ от всех фактов обо всех обитателях острова, известных наблюдателю.

Предложим простой способ классификации метрик ситуационной осведомленности (далее - метрики СО).

— Статические метрики SA – метрики, описывающие состояния сцены и объектов;

— Метрики SA состояний – метрики, описывающие осведомленность о состояниях объектов;

— Метрики SA событий – метрики, описывающие факты, описывающие действия.

Второй подход к классификации метрик SA опирается на три стадии SA по М.Эндсли.

Метрики восприятия M^P , под которым обычно понимают сбор и обработку данных о текущем состоянии системы.

Метрики осмысления (анализа) M^A — интерпретация метрик системы, оценка их значимости.

Метрики прогнозирования M^F — предсказание будущих состояний системы на основе текущего состояния и истории динамики системы.

В качестве примера можно привести некоторое количество метрик ситуационной осведомленности и применить к ним предложенную классификацию (таблица 1.1).

Таблица 1.1 — Классификатор метрик SA

ID	I	M^P	M^A	M^F
M_1	Количество известных фактов, отнесенное к общему числу	+	+	-
M_2	Процент пропущенных существенных фактов	+	+	-
M_3	Время оценки ситуации наблюдателем с заданным уровнем точности	+	+	+
M_4	Точность возможного прогнозирования ситуации	-	-	+
M_5	Вероятность возможной проверки гипотез в будущем. Например: "Какова вероятность определения хозяина зебры при $T = 220$	-	-	+
M_6	Глубина возможного прогнозирования	-	-	+

M_1 – количество известных фактов, отнесенное к общему числу:

$$M_1(T_k) = \tag{1.4}$$

M_2 – процент пропущенных существенных фактов;

M_3 – время оценки ситуации наблюдателем с заданным уровнем точности;

M_4 – точность возможного прогнозирования ситуации.

M_5 – устойчивость оценки ситуации к пропускам фактов.

M_6 – глубина возможного прогнозирования.

2 Постановка задачи на примере Zebra Puzzle

Задача Zebra Puzzle (также известная как "Эйнштейновская головоломка") была впервые опубликована в журнале ”Life” [3] представляет собой классическую логическую задачу, в которой необходимо определить правильное расположение атрибутов в ряде домов на основе заданных подсказок.

Мы будем использовать предложенную нами расширенную версию этой задачи, включающую 6 домов (House1, House2, House3, House4, House5, House6). (рисунок 2.1).

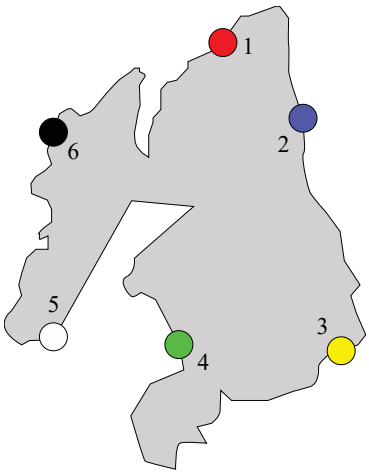

Рисунок 2.1 — Остров для модифицированной ZEBRA PUZZLE

Дома покрашены в 6 цветов (Red, Blue, Yellow, Green, White, Black).

В домах живут островитяне 6 национальностей (Russian, English, Chinese, German, French, American).

Каждый островитянин описывается комбинацией следующих атрибутов:

— один напиток из 6 возможных (Water, Beer, Juice, Whiskey, Vodka, Wine),

— одна марка сигарет из 6 возможных (Marlboro, PallMall, Dunhill, Kent, Camel, Parlament),

— один питомец из 6 видов животных (Dog, Cat, Zebra, Fish, Hamster, Bear).

Для того чтобы проанализировать влияние поведенческой модели наблюдателя на его SA рассмотрим две поведенческих модели.

В первом сценарии будем считать, что каждый обитатель острова, находящийся дома, в соответствии с некоторым алгоритмом, выбирает место для поездки в гости к одному из соседей. Поездка в одну сторону занимает несколько дней. При этом если в день завершения поездки хозяина дома нет, то информация об обстановке в доме не становится доступной гостю и гость просто возвращается домой.

Во втором сценарии факт владения дома островитянином просто означает привязку дома к владельцу, но не обязывает его возвращаться домой при перемещениях по острову.

В первом сценарии в каждом доме всегда ночует не более одного островитянина.

Независимо от выбранного сценария, при встрече нескольких обитателей острова, с некоторой вероятностью, может происходить обмен питомцами и/или домами. Алгоритмы обмена могут быть разными. Каждый островитянин самостоятельно принимает решение об участии в обмене. Для того, чтобы обмен состоялся достаточно двух участников. Сам процесс обмена может быть описан некоторым алгоритмом. Простейшим примером такого алгоритма является случайный выбор.

Считаем что проверка фактов и состояний производятся в день завершения поездки. То есть если я сегодня стартовал и поездка занимает два дня то послезавтра утром я оказываюсь в месте назначения. При этом я застану хозяина (при выборе первого сценария) или всех, кто ночевал в доме и начинает путешествие в этот день.

При этом, при реализации первого сценария, в том случае, если в день завершения поездки гостя, хозяина дома нет, то информация об обстановке в доме не становится доступной гостю и гость просто начинает поездку обратно домой.

Таблица расстояний 2.1 содержит расстояния между домами, измеренное в днях пути, использованное в [4] и альтернативный граф, в котором разрешены все перемещения.

Таблица 2.1 — Таблица расстояний (в днях) между домами (пример простой симметричной схемы-кольца **A** и полного графа **B**)

A	Color	1	2	3	4	5	6
1	Red	NA	1	NA	NA	NA	1
2	Blue	1	NA	1	NA	NA	NA
3	Yellow	NA	1	NA	1	NA	NA
4	Green	NA	NA	1	NA	1	NA
5	White	NA	NA	NA	1	NA	1
6	Black	1	NA	NA	NA	1	NA

B	Color	1	2	3	4	5	6
1	Red	7	10	2	4	5	3
2	Blue	7	4	5	8	7	9
3	Yellow	8	7	1	8	1	3
4	Green	3	4	6	2	6	8
5	White	7	10	7	4	1	8
6	Black	8	2	4	10	5	7

Несложно заметить, что в первом случае все перемещения ограничены возможностью походов в гости к соседу слева и соседу справа и для расстояний верно, что $|a,b| = |b,a|$.

Для сравнения мы рассмотрим второй вариант полного направленного графа, для которого $|a,b| \neq |b,a|$.

Информация об участниках встречи и их питомцах становится известной всем участникам встречи и является достоверной.

Цель каждого из участников – в каждый момент времени иметь информацию, кто где живет, что пьет, курит и какое животное держит, а также текущее состояние обстановки (информацию о том, кто и куда ходил в гости, где сейчас находится и какие обмены имели место), используя информацию, которая стала им известна лично или из публикаций в бюллетенях. При этом какие именно метрики ситуационной осведомленности будут использованы определяются на этапе постановки задачи и это существенно влияет на стратегии островитян.

Полный список типов событий в системе приведен в таблице 2.2.

Таблица 2.3 содержит вероятности, используемые при моделировании выбора островитянами в случае когда в качестве механизма выбора используется случайный выбор с некоторыми вероятностями.

Здесь:

— ID – код островитянина;

— I – национальность;

Таблица 2.2 — Полный список типов событий

ID	I	1	2	
e_1	$StartTrip(ID,D,I,H1,R)$	Начало путешествия		
	ID	код события		
	D	день		
	I	код островитянина		
	$H1$	код дома из которого производится поездка		
	$H2$	код дома в который производится поездка		
e_2	FinishTrip(ID,D,event,StartID, I,R)	Завершение путешествия		
	ID	код события		
	D	день		
	$event$	тип события		
	$StartID$	код события StartTrip		
	I	код островитянина		
	R	результат поездки: 1-успешно, 0 - никого нет дома		
e_3	changeHouse(ID,D,N,I1,I2,I3, H1,H2,H3)	Обмен домами		
	ID	код события		
	D	день		
	N	число участников обмена		
	$I1$	первый участник обмена		
	$I2$	второй участник обмена		
	$I3$	третий участник обмена		
	$H1$	дом первого участника после обмена		
	$H2$	дом второго участника после обмена		
	$H3$	дом третьего участника после обмена		
e_4	changePet(I1,I2,I3,P1,P2,P3)	Обмен питомцами		
	ID	код события		
	D	день		
	N	число участников обмена		
	$I1$	первый участник обмена		
	$I2$	второй участник обмена		
	$I3$	третий участник обмена		
	$P1$	питомец первого участника после обмена		
	$P2$	питомец второго участника после обмена		
	$P3$	питомец третьего участника после обмена		

— 1-6 – вероятность о принятии решения о поездке в дом соответствующего цвета;

— PHouseExch– вероятность готовности к обмену домами;

— PPetExch– вероятность вероятность готовности к обмену питомцами.

Результат моделирования представляет собой LOG файл содержащий набор событий:

```
eventID;day;event;...
1;1;startTrip;Russian;1;2
```

Таблица 2.3 — Таблица вероятностей для выбора маршрута островитянами

ID	I	1	2	3	4	5	6	PHouseExch	PPetExch
1	Russian	100	0	0	0	0	0	70	70
2	English	30	10	10	25	25	0	50	50
3	Chinese	30	20	20	20	20	0	50	50
4	German	50	0	0	0	0	50	25	25
5	French	10	10	0	10	20	50	90	40
6	American	15	15	15	15	20	20	100	100

```
2;2;startTrip;Chinese;3;2
3;3;FinishTrip;1;Russian;1
4;3;FinishTrip;2;Chinese;1
5;3;changePet;3;Russian;Englishman;Chinese;Zebra;Cat;Dog
6;3;startTrip;Russian;2;1
7;3;startTrip;Chinese;2;3
8;4;FinishTrip;7;Chinese
9;5;FinishTrip;6;Russian
...
```

В приведенном примере в первый день русский из первого дома начинает двухдневную поездку во второй дом. Наглядное представление перемещений приведено на циклограмме (рисунок 2.2). Во второй день китаец из третьего дома начинает однодневную поездку во второй дом.

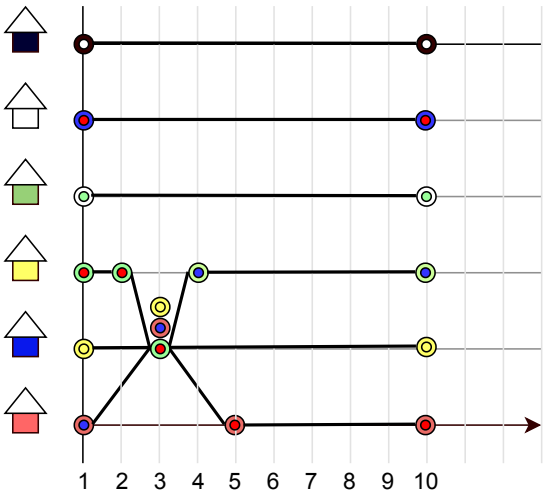

Рисунок 2.2 — Пример циклограммы, описывающей перемещения по острову

На третий день русский и китаец прибывают во второй дом и застают там англичанина. Происходит обмен питомцами в результате которого у русского появляется зебра, у англичанина кошка и у китайца - собака. В этот же день русский и китаец стартуют в обратное путешествие.

2.1 Постановка задачи оптимальной стратегии агентов

В рамках рассмотренной задачи становится возможным вопрос о том, какой стратегии должен придерживаться активный наблюдатель (в нашем случае островитянин), чтобы иметь максимальную осведомленность о ситуации на острове. При этом сама осведомленность может быть оценена по разному.

Будем рассматривать две метрики осведомленности:

а) M_1^t - количество известных фактов, отнесенное к общему числу фактов в момент времени t.

б) M_2^t - индикаторная функция для утверждения "Мне известно, кому принадлежит зебра в момент времени t".

Также заметим, что возможны различные способы агрегировать значения метрик осведомленности за все ходы. Будем называть это функцией сбора и обозначать, как $F_x(M_a, n)$, где M_a - выбранная метрика осведомленности, n - общее число ходов.

Рассмотрим несколько возможных функций сбора:

а) $F_1(M_a, n) = \sum\limits_{i=1}^{n} M_a^i$ - то есть сумма метрик осведомленности за все ходы.

б) $F_2(M_a, n) = M_a^n$ - то есть метрика осведомленности на последнем ходу.

в) $F_3(M_a, n) = \sum\limits_{i=1}^{n} \frac{i}{n} M_a^n$ - схожа с F_1, но учитывает новые результаты сильнее, чем старые.

Таким образом, оптимальная стратегия агента A при зафиксированных (F_x, M_a, n) представляет собой такую стратегию, что при фиксированных действиях других агентов $F_x(M_a, n)$ будет максимальным среди значений для всех возможных стратегий игрока A.

3 Постановка задачи на разработку тактической игры

Разработка прототипа симулятора для тактических игр на примере Zebra Puzzle

Игру реализуем с простым прототипом интерфейса

Количество игроков от 3 до 20

Каждый из игроков может быть как компьютерным, так и реальным.

Задачи по участникам проекта разбиваются следующим образом:

1) Нуриддин Шарипов (НШ)

Разработка базовой технологической платформы (сервер + Telegram Bot, сессии/лобби, коммуникация)

2) Дододжон Гоибов (ДГ)

Разработка решения для симуляции задачи (ядро симулятора, логирование, модель знаний и событий)

3) Мухаммадали Темуров (МТ)

Разработка программной реализации алгоритма действия игрока (стратегии/оптимизация для максимизации SA, интеграция через API)

3.1 Разработка базовой технологической платформы

Базовая технологическая платформа должна обеспечивать моделирование системы для произвольного числа участников (от 3 до 20) и поддерживать смешанный состав игроков: реальные участники и компьютерные агенты.

Клиентское приложение должно быть реализовано в виде Telegram Bot и предоставлять простой прототип игрового интерфейса: создание комнаты (лобби), подключение участников, приглашение по тегам, старт партии и пошаговый выбор действий игроками. Те участники группы, которые не стартуют сессию или не подтверждают участие в течение некоторого фиксированного времени TimeDelay, заменяются симуляторами (компьютерными агентами) и далее совершают автоход.

Сервер должен публиковать REST API для:

— создания игровой сессии и конфигурации параметров (число игроков, число домов, длительность партии, режим графа перемещений ring/full, режим взаимодействий);

— приема действий реальных игроков (выбор действия в текущий день);

— выполнения шага симуляции (один день) и выдачи промежуточного состояния (день, список ожидаемых ходов, текущие значения метрики M_1);

— принудительного завершения партии и получения итоговых результатов.

Результат игры приводится в виде CSV и XML лог файла, а также в виде файла метрик, содержащего значения ситуационной осведомленности $M_1(t)$ для каждого островитянина по дням. Telegram Bot должен получать от сервера эти результаты и отправлять их пользователям в виде набора файлов/архива.

Обработка LOG файла производится третьей компонентой программы и включает: преобразование логов в удобный формат (например, YAML/CSV-таблицы), извлечение метрик $M_1(t)$ и подготовку данных для построения графиков.

3.2 Разработка решения для симуляции задачи

Симулятор должен работать с сервером без использования Telegram Bot и обеспечивать полный цикл моделирования по заданной конфигурации.

Симулятор реализует динамику игры по дням и поддерживает:

— перемещение агентов между домами согласно графу (ring/full) с учетом времени в пути;

— обмен информацией при встречах (в зависимости от режима взаимодействий), обновление знаний агента о признаках (цвет, национальность, напиток, питомец, сигареты);

— вероятностные события (например, шум/потери информации) при необходимости;

— обработку действий игроков: если игрок не сделал ход за TimeDelay, используется автоход (стратегия компьютерного агента).

Симулятор формирует логи и метрики:

— событийный лог CSV (последовательность событий по дням),
— событийный лог XML (тот же лог в XML),
— таблицу метрик $M_1(t)$ по каждому агенту за каждый день.

3.3 Разработка программной реализации алгоритма действия игрока

Программная компонента оптимизации действий игрока должна взаимодействовать с API сервера и выбирать стратегию игрока для достижения максимальной ситуационной осведомленности.

Стратегия игрока задается параметрами (например, распределение вероятностей перемещения по домам и вероятности обменов) и хранится в YAML. Компонента оптимизации должна:

— запускать серии симуляций через REST API;
— оценивать качество стратегии по целевой функции (например, максимизация итогового M_1 или среднего M_1 на хвосте времени);
— выбирать лучшую стратегию и сохранять ее в виде YAML-конфига, который затем может использоваться для автоходов или для сравнения с действиями реальных игроков.

3.4 Разработка решения для анализа результатов моделирования

Необходимо реализовать построение графика зависимости ситуационной осведомленности от времени для каждого островитянина.

Графики строятся на основе файла метрик $M_1(t)$, полученного из симулятора/логов. Результат анализа должен включать:

— график $SA(t) = M_1(t)$ для всех игроков (несколько кривых на одном графике либо набор графиков),

— при необходимости таблицы/сводки итоговых значений (например, M_1 в последний день) и ранжирование игроков.

В качестве инструмента можно использовать либо Python Shiny либо XML FO (допускается реализация на Python с генерацией графиков и сохранением в файлы для отчета).

3.5 Архитектура и компоненты системы

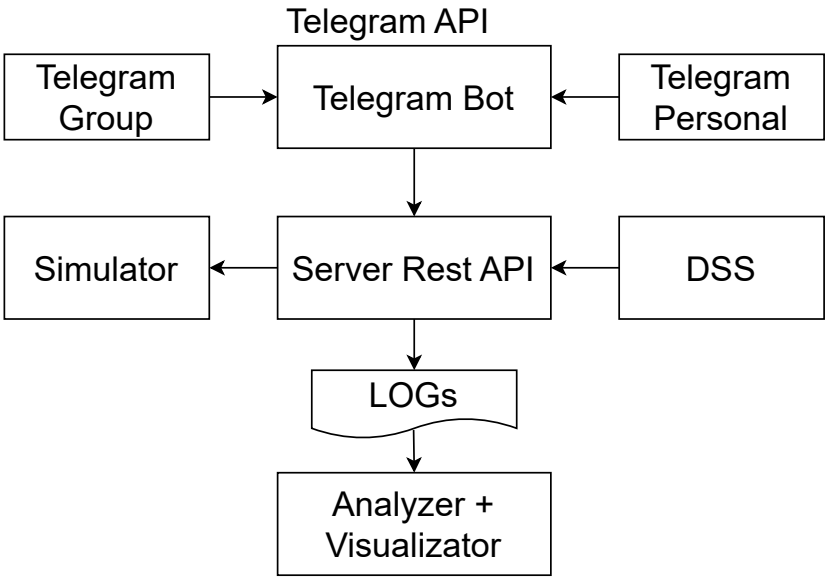

Рисунок 3.1 — Архитектура решения для Telegram]

Система включает четыре ключевые части (рис.3.1):

— Telegram Bot (интерфейс)
— FastAPI Server (REST API)
— Simulator (ядро моделирования)
— Analysis/Optimizer (постобработка и алгоритм действий)

3.6 Реализованный функционал на текущий момент

Текущее состояние работ по проекту приведено в таблице 3.1.

Таблица 3.1 — Реализованный функционал на текущий момент

Компонента	Реализовано
Сервер (REST API)	создание/ведение игры, состояния, действия, шаг, завершение; health-check
Симулятор (batch)	прогон на T дней, CSV+XML лог, граф домов ring/full, встречи, шум.
Интерактивная игра (step-by-step)	ожидание ходов людей, автодействия ботов, согласованные обмены питомцами, отчеты по дню.
Telegram Bot (интерфейс)	лобби, join/leave, приглашение, приватные ходы, динамические кнопки, завершение/голосование.
Стратегии/оптимизация (MT)	Частично: инфраструктура стратегий, метрики и точка интеграции; требуется доработка логики оптимизации и эксперименты.
Анализ результатов	обработка metrics, построение графиков SA(M1) от времени; Shiny/FO - в плане.

4 Обзор литературы

4.1 История и современные представления о SA

В работе [5] ситуационная осведомленность определяется как восприятие сущностей в окружающей обстановке, понимание их значения и прогнозирование их состояния в ближайшем будущем. Авторами предложена модель для SA и динамического принятия решений, которая объединяет искусственный интеллект и динамические системы, управляемые данными.

Для адаптации измерений и ресурсов в соответствии с изменяющейся обстановкой. авторами ставится проблема измерения SA и проблемы, связанные с ее количественной оценкой. Далее мы подробно рассматривается множество методов и технологий, которые способствуют повышению SA. Рассмотрены разные подходы от различных методов сбора разведданных до искусственного интеллекта и автоматизированных систем компьютерного зрения. Также в обзоре представлены различные области применения SA, включая поле боя, “серую зону” гибридной войны, военные и авиационные базы, безопасность и оборону, а также критическую инфраструктуру.

Работы [6, 7] посвящены обсуждению проблемы взаимодействия по схеме ”человек в контуре” (HITL), когда операторы-люди влияют на исход миссии. При этом авторы используют моделирование для поддержки боевых действий. В статье приведены оценки эффективности действий операторов-людей и их взаимодействия со средой моделирования. Объективное измерение ситуационной осведомленности вызывает значительный интерес, поскольку эти знания имеют решающее значение для процессов успешного принятия решений (C2). Авторы используют когнитивно-вычислительный подход для измерения ситуационной осведомленности, основанный на теории моделей ситуации. Измерения метрик СО в рамках этого подхода отражают информацию о сущностях, пространственных и временных параметрах, целях участников и причинно-следственных связях событий. При этом внутри наблюдаемой системы выде-

ляются Situation Awareness Objects (SAOs), информация об изменениях состояний которых и формирует ситуационную осведомленность.

Задача решается путем логических заключений но эта цепочка является достаточно длинной. В ряде статей приведены подходы к решению этой задачи на основе формальных методов булевой алгебры (см., например, [8]).

После некоторого количества неудач в итоге появился ряд подходов, успешно справляющихся с этой задачей с применением различных специальных средств ([9] – решение на Prolog, [10] – моделирование в терминах логики первого порядка)

4.2 Использование ZEBRA PUZZLE для модельных задач

Классические LLM решения логические задачи не решают. Сложные ветвистые рассуждения не могут быть корректно исполнены и верифицированы.

В работе [11] представлена многозадачная система ZPS, которая интегрирует LLM с готовым решением для FOL. Эта система решает сложную задачу разгадывания головоломок, разбивая проблему на более мелкие, управляемые части, генерируя SMT-код (Satisfiability Modulo Theories) и используя обратную связь между агентами для многократного улучшения их ответов.

4.3 Варианты решений для решателей включая FOL

MiniSat - классический компактный решатель

Glucose - продвинутая версия MiniSat

Z3 - открытый и мощный решатель от Microsoft Research

Clingo - решение на Prolog

CVC5 - приличный солвер (стоит пощупать примеры задач) [12]

Vampire - автоматический солвер для теорем первого порядка

MiniZinc - декларативное моделирование ограничений

ALLOY - хорош для проектирования модели данных, верификации инвариантов системы и генерации тестовых сценариев

4.4 Теоретико-игровой подход к задаче SA

Авторы в статье [13] приводят обзор многочисленных публикаций, посвященных применению теории игр для решения задач осведомленности и оценки объектов, ситуаций, сетей и пользователей.

Они систематизируют применения теории игр в области оценки обстановки и осознания ситуации, выстраивая четыре варианта использования по аналогии с уровнями информационного слияния: от сенсорных данных к анализу ситуации, управлению знаниями и, в конечном итоге, к взаимодействию с пользователем. В качестве примера они описывают сценарий, в котором множество распределенных участников в сетях одновременно участвуют в иерархии взаимосвязанных игр, затрагивающих данные, информацию, знания и социальные аспекты.

4.5 Обзор игровых симуляторов

Игровые симуляторы в прикладных инженерных и организационных задачах занимают промежуточное положение между натурным экспериментом и аналитической моделью: они позволяют воспроизвести существенные свойства реальной системы в управляемой среде, сохранив повторяемость сценариев и наблюдаемость ключевых событий. Для задач ситуационной осведомленности это особенно важно, потому что SA проявляется не как статический “объем данных”, а как процесс: участник по ходу игры уточняет картину мира, распределяет внимание, проверяет гипотезы и превращает наблюдения в действия [14].

Практика разработки serious games показывает, что исследовательская ценность симулятора заметно растет, когда игра проектируется не только как обучающий инструмент, но и как средство получения данных о поведении. Тогда заранее фиксируются измеряемые величины и правила протоколирования: какие решения принимал игрок, что именно было ему доступно в момент выбора, какие события изменили ситуа-

цию и как эти изменения повлияли на дальнейшие действия. Такой подход помогает интерпретировать динамику SA как следствие конкретных игровых эпизодов, а не как общую субъективную оценку “успешности” стратегии [14].

Отдельный интерес представляют короткие игровые форматы, рассчитанные на быстрый цикл обратной связи. В них сохраняется структура компромиссов, характерная для сложных систем: локальное решение способно изменить глобальную обстановку, а улучшение одного показателя может сопровождаться ухудшением другого. При этом короткая сессия делает изменения осведомленности наблюдаемыми на малом горизонте: игрок видит связь между выбором и последствиями уже на следующем шаге. В литературе этот класс подходов описывается через концепцию microgames - сокращенной формы симуляционной игры, сфокусированной на конкретной проблеме заказчика и предназначенной для гибкого обучения в динамичных условиях [15].

В рамках проекта Zebra Puzzle указанные принципы реализуются в виде клиент-серверного игрового сервиса. Telegram Bot используется как прототип интерфейса, который поддерживает диалоговый игровой процесс: формирование лобби, подключение участников, выдачу актуального состояния и выбор действий на каждом ходе. Серверная часть публикует REST API и хранит единственный источник истины о состоянии сессии, выполняя пошаговую дискретно-событийную симуляцию. Такое разделение сохраняет “игровость” на уровне интерфейса (решение игрока и немедленная обратная связь), но при этом обеспечивает воспроизводимость экспериментов и контролируемость правил мира, что критично для последующего анализа SA [14].

С точки зрения анализа результатов ключевым элементом игрового симулятора остается протоколирование. Логи событий и состояния позволяют восстановить последовательность шагов и сопоставить ее с динамикой метрик ситуационной осведомленности, а значит - сравнивать стратегии поведения при одинаковых параметрах среды и выявлять, какие именно классы действий дают наибольший вклад в рост SA во времени [14].

4.6 Обзор DES решений для задач SA

Центральной проблемой при оптимизации дискретной модели является управление FES, описываемым очередью Q_F, и представляющим собой вычислительное ядро любого дискретного симулятора. От эффективности операций вставки, удаления и выбора следующего события напрямую зависят общая производительность и масштабируемость моделей, особенно при работе с большими объемами данных. Ранние работы в этой области были посвящены поиску и сравнительному анализу оптимальных структур данных для реализации FES. Исследование W.R.Franta и K.Maly "A comparison of heaps and the TL structure for simulation event set" [16] является типичным примером этого этапа. В нем авторы проводят систематическое сравнение двух структур данных, пирамид (heaps) и TL-структур, демонстрируя их преимущества и недостатки с точки зрения вычислительной сложности и производительности на практике.

Параллельно с исследованиями фундаментальных алгоритмов активно развивались специализированные языки имитационного моделирования, призванные абстрагировать исследователя от низкоуровневых деталей реализации и предоставить ему удобный инструмент для описания сложных систем. Ранним примером такого подхода является язык SOL, представленный и формально описанный в 1964 году в работах D.E.Knuth и J.L.McNeley [17, 18].

Алгоритм ленивой очереди основан на идее минимизации количества операций переупорядочивания списка, выполняя их только в момент, когда действительно требуется выбрать следующее событие. Это позволяет снизить накладные расходы в ситуациях, когда временные метки событий имеют специфические паттерны.

Динамическая календарная очередь, в свою очередь, нацелена на решение проблемы выбора оптимального размера "календаря" — параметра, критически влияющего на производительность классического алгоритма календарной очереди. Она динамически подстраивает свои внутренние параметры в процессе моделирования, в ответ на изменения плотности и распределения событий во времени.

Представленный обзор позволяет четко проследить две параллельные и взаимовлияющие линии развития имитационного моделирования. Первая линия — это углубленное, изучение фундаментальных структур данных и алгоритмов, обеспечивающих работу ядра любой имитационной системы — механизма управления событиями.

Вторая линия — это развитие инструментов имитационного моделирования, т.е. языков и сред.

Хорошим системным изложением существующих подходов к дискретно – событийному моделирования является работа [19].

5 Описание решения

5.1 Простая модель поведения

Простая модель поведения – бросание монетки всякий раз, кода островитянин оказывается перед выбором.

В начале хода он выбирает из трех вариантов

а) сходить в гости;

б) остаться дома.

Если в одном месте собираются два или более островитян, каждый принимает решения:

- меняться ли домами

- меняться ли питомцами.

При этом мы рассматриваем два сценария

1) Мы считает известным островитянину только тот набор фактов, свидетелем которых он был.

2) Мы считаем, что островитяне при встрече обмениваются информацией, которая им доступна.

5.2 Метрики ситуационной осведомленности

Всего система в момент времени k описывается конечным набором фактов $\mathcal{F}^k = \{f_1^k, \ldots, f_W^k\}$.

В простейшем случае ситуационная осведомленность n - го островитянина d момент времени t^k может быть измерена как отношение числа известных ему фактов к полному набору фактов:

$$\mathcal{G}_n = |\mathcal{F}_e^n|/|\mathcal{F}_e| \tag{5.1}$$

Типы фактов в системе по состоянию на некоторый момент времени:

Островитянин B_n^k живет в доме h.

Островитянин B_n^k пьет напиток d.

Островитянин B_n курит сигареты s.

Островитянин B_n^k владеет питомцем p.

Легко заметить, что число фактов, описывающих систему в момент времени T^k :

$$|\mathcal{F}_e| = N \times 4 = 24 \tag{5.2}$$

Значительно сложнее оценить осведомленность конкретного n-го островитянина.

в начальный момент времени $t = 0$ он знает только собственный набор $h_n^0, d_n^0, s_n^0, p_n^0$

5.3 Метрики ситуационной осведомленности в простейшем случае

В простейшем случае мы считаем что островитяне не обмениваются информацией и им известны только факты, свидетелями которых они были.

При этом есть еще общая информация, которая не меняется во времени и влияет на SA островитян:

а) информация о количестве островитян;
б) информация о порядке расположения домов по кругу.

5.4 Метрика "У кого зебра"

Интересным частным случаем метрики SA может быть например осведомленность о том, в каком доме живет зебра.

Понятно , что владельцу зебры невыгодно ей меняться. Но это не мешает нам порассуждать и формально поставить задачу SA и выбора метрики SA для этого случая.

6 Результаты

6.1 Описание архитектуры игрового сервиса

Игровой сервис реализован как клиент-серверная система. Telegram Bot выступает интерфейсом и “ведущим” игры: он показывает лобби, принимает решения игроков и следит за тем, чтобы ход дня завершался только после того, как собраны действия. Сервер хранит единственный источник истины о состоянии сессии и выполняет дискретно-событийную симуляцию. Такое разделение позволяет независимо развивать пользовательские сценарии, модель мира и обработку результатов, не ломая весь проект при изменениях в одной части.

6.1.1 Состав компонент

Система состоит из нескольких компонент, каждая из которых решает свою задачу и при этом остается достаточно автономной.

Telegram Bot отвечает за пользовательский опыт. Он работает сразу в двух каналах: групповой чат используется как общая сцена (лобби, объявления о старте, статус ожидания, финальные результаты), а личные сообщения - как приватный интерфейс для хода конкретного игрока. В личке игрок получает максимальную “ситуационную картину” на текущий день: где он находится, что известно о домах, какие действия доступны сейчас. Кнопки формируются динамически и зависят от состояния: если игрок в пути, выбор действий один; если игрок в доме и есть варианты перехода, появляются другие кнопки.

REST API сервер является управляющей частью. Через него создается игра, запрашиваются глобальные и индивидуальные состояния и регистрируются действия участников. На сервере сосредоточена логика корректности: именно он решает, можно ли выполнить действие в текущем состоянии, и именно он обновляет состояние игры при переходе к следующему дню. Таким образом, бот не “симулирует” мир, а только отображает его и передает решения игроков.

Симуляционное ядро реализует правила мира как дискретно-событийную модель: перемещения по графу домов, встречи и посещения, на-

блюдения, обмены (если они разрешены правилами) и обновление знаний. Здесь же пересчитываются метрики ситуационной осведомленности по шагам.

Для экспериментов предусмотрен автономный режим прогона: симуляция может работать без Telegram Bot, обращаясь к серверу через те же эндпоинты, что и интерфейс. Это нужно для пакетных запусков, тестирования сценариев и сравнения стратегий в одинаковых условиях.

Компонента стратегии/оптимизации рассматривается как отдельный слой, который также взаимодействует с API. В рамках проекта она нужна для реализации поведения компьютерных игроков и для алгоритма выбора действий (роль MT), ориентированного на рост ситуационной осведомленности.

Наконец, компонент анализа читает сформированные логи и строит графики зависимости SA(t) для каждого участника. В результате можно сравнивать стратегии и проверять, какие действия и взаимодействия дают вклад в рост осведомленности.

6.1.2 Интерфейсы и обмен данными

Взаимодействие организовано по двум основным линиям. Первая линия - Telegram API (через aiogram): доставка сообщений, обработка нажатий кнопок и разделение коммуникации между группой и личными сообщениями. Вторая линия - HTTP REST: обмен между ботом/стратегиями и сервером, включая создание сессии, получение состояния, регистрацию действий и выполнение шага моделирования.

При реализации приглашений учитывается важное ограничение Telegram: бот может написать пользователю в личные сообщения только если пользователь ранее начал диалог (например, нажал `/start`). Поэтому в личку можно отправлять приглашения и приватные ходы только тем игрокам, чьи идентификаторы уже известны системе.

6.1.3 Жизненный цикл игры

Жизненный цикл сессии начинается с лобби. Хост создает лобби в групповом чате и задает параметры игры: число игроков, число домов, длительность, режим обмена знаниями, наличие шума наблюдений и тип графа перемещений. Далее запускается ожидание до момента `TimeDelay`. Игроки присоединяются через кнопку Join; те, кто не успел, могут быть заменены симуляторами.

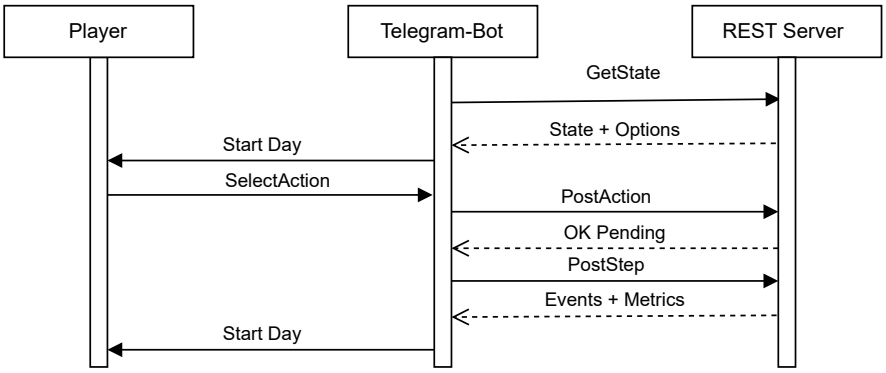

Рисунок 6.1 — Жизненный цикл решения для Telegram]

После старта бот формирует запрос на сервер для создания игры. Сервер инициализирует модель: дома, связи между домами, стартовые атрибуты и базовые знания игроков. Если реальных участников меньше требуемого числа, недостающие роли заполняются компьютерными агентами, чтобы игра была полноценной и симуляция оставалась корректной.

Далее игра идет по дням. В начале дня бот запрашивает у сервера список игроков, от которых требуется действие, и каждому реальному игроку отправляет в личку индивидуальное состояние вместе с допустимыми действиями. Игрок выбирает действие, бот передает его на сервер. Когда действия собраны (или срабатывает таймер хода), бот инициирует шаг `step`: сервер применяет решения, обновляет перемещения и события, пересчитывает знания участников и метрики SA. После этого бот рассылает игрокам уведомления по итогам дня (например, кто приходил в дом, какие факты стали известны, что изменилось) и публикует в группе краткий статус прогресса.

Завершение игры происходит при достижении последнего дня, при принудительном завершении или по коллективному решению. Если активен только один реальный игрок, игра завершается сразу. Если реальных игроков больше одного, бот запускает голосование в группе, и при большинстве голосов сервер переводит игру в финальное состояние через `/game/{gid}/finish`. После завершения выводится итоговая таблица результатов и выгружаются файлы логов.

6.1.4 Механика знаний и событий, влияющих на SA

Ситуационная осведомленность в модели формируется через наблюдения и взаимодействия. Когда игрок находится в доме, он получает факты о доме и связанных с ним атрибутах. Если несколько участников оказываются в одном доме, возникает событие встречи, и в зависимости от режима `share` может происходить обмен знаниями. Обмены (например, обмен питомцами) допускаются только при выполнении условий совместного присутствия и при согласии участников, то есть реализуется двусторонняя схема подтверждения. При включенном параметре `noise` часть наблюдений может искажаться, поэтому рост SA зависит не только от перемещений, но и от стратегии проверки и сопоставления фактов.

6.1.5 Логирование и подготовка данных для анализа

По окончании сессии сервер формирует лог-файлы в форматах CSV и XML. CSV отражает последовательность событий по дням (перемещения, встречи, обмены и служебные переходы) и позволяет восстановить «сюжет» игры как цепочку причин и последствий. XML содержит структурированное представление событий с параметрами, удобное для дальнейшей машинной обработки. Отдельно формируются временные ряды метрик SA по участникам в CSV, на основе которых строятся графики SA(t). Это дает возможность сравнивать стратегии поведения и анализировать вклад отдельных действий и взаимодействий в изменение ситуационной осведомленности.

Заключение

В результате выполненного этапа собран и запущен прототип игрового сервиса по мотивам Zebra Puzzle, предназначенный для моделирования и оценки ситуационной осведомленности участников в пошаговой игре. Сервис реализован в клиент-серверной форме: Telegram Bot обеспечивает взаимодействие с игроками и ведение сессии, а серверная часть хранит состояние игры и выполняет дискретно-событийную симуляцию по дням с формированием логов.

В результате работы решены следующие задачи:

— реализована базовая архитектура игрового сервиса (Telegram Bot + REST сервер) и сквозной запуск игровой сессии

— реализован игровой цикл по дням: сбор действий, выполнение шага симуляции, выдача игрокам персональной обратной связи и публикация статуса в группе

— реализовано логирование результатов моделирования в форматах CSV и XML как основа для дальнейшего анализа SA(t)

— реализованы базовые сценарии завершения сессии, включая принудительное завершение при единственном живом игроке и завершение по голосованию при наличии нескольких живых игроков

План работ по игровому сервису на следующем этапе связан не с расширением списка функций, а с доведением игрового процесса до состояния, в котором решения игрока становятся центральным элементом симуляции. В первую очередь требуется стабилизировать ход: игрок должен видеть полную картину текущего дня, понимать, где он находится и куда направляется, и иметь только те кнопки, которые действительно допустимы в его ситуации. Отдельное внимание планируется уделить механике перемещений, чтобы состояние “в пути” корректно сокращалось и однозначно приводило к прибытиям, а выбор направления или целевого дома был прозрачным для игрока.

Вторая линия работ - взаимодействия между участниками. В текущем прототипе заложены встречи и обмены, но для “настоящей игры” важно формализовать согласие: обмен должен предлагаться только при

совместном присутствии в доме и завершаться только после подтверждения обеих сторон с обязательными уведомлениями. Это позволит, с одной стороны, сохранить причинность событий, а с другой - сделать коммуникацию источником роста ситуационной осведомленности.

Третья линия работ относится к привлечению и подключению живых игроков. Основная сцена остается в группе, но личные сообщения используются для приватных ходов и индивидуальной информации. Планируется довести механику приглашений по Telegram-тегам до устойчивого сценария, учитывающего ограничения платформы (возможность писать в личку только пользователям, которые ранее инициировали диалог с ботом). Это необходимо, чтобы реальная многопользовательская сессия собиралась без ручных действий и оставалась удобной с мобильных устройств.

Завершающая часть плана - аналитический контур. На следующем этапе требуется оформить обработку логов и построение графиков SA(t) как отдельную воспроизводимую процедуру, а также подготовить набор демонстрационных прогонов на одинаковых настройках. Это даст возможность сравнивать стратегии игроков и показать, как действия и взаимодействия влияют на динамику ситуационной осведомленности в ходе игры.

Список использованных источников

1. *Endsley, Mica R.* Toward a theory of situation awareness in dynamic systems / Mica R Endsley // *Human factors.* — 1995. — Vol. 37, no. 1. — Pp. 32–64.

2. Issues and Challenges in Situation Assessment (Level 2 Fusion) / Erik Blasch, Ivan Kadar, John S. Salerno et al. // *J. Adv. Inf. Fusion.* — 2006. — Vol. 1. — Pp. 122–139. `https://api.semanticscholar.org/CorpusID:1805918`.

3. Are You a Logical Thinker? Here's a Test // *Life International.* — 1962. — Vol. 33, no. 13. — P. 92. — The original publication of the Zebra Puzzle. The solution was published in the March 25, 1963, issue.

4. Event-Based Simulation with Active Objects and Dynamic Scheduling / I.V. Ablov, T.S. Afanasenko, A.V. Khelvas et al. // *Information Processes.* — 2026. — 01.

5. *Munir, Arslan.* Situational Awareness: Techniques, Challenges, and Prospects / Arslan Munir, Alexander Aved, Erik Blasch // *AI.* — 2022. — Vol. 3, no. 1. — Pp. 55–77. `https://www.mdpi.com/2673-2688/3/1/5`.

6. *Curiel, Jacqueline.* An Interdisciplinary Approach to the Study of Battlefield Simulation Systems / Jacqueline Curiel, Jacob Tran, Ke-Thia Yao // *Proceedings of the Interservice / Industry Training, Simulation and Education Conference.* — 2004. — 01.

7. Developing Situation Awareness Metrics in a Synthetic Battlespace Environment / Jacqueline Curiel, Michael Anhalt, Jacob Tran, Ke-Thia Yao // *Interservice/Industry Training, Simulation, and Education Conference (I/ITSEC) 2005.* — 2005. — 01.

8. *Goliney, Igor.* Solving zebra puzzle with Boolean algebra. — 2015. — 07.

9. *Gregor, Michal.* The zebra puzzle and getting to know your tools / Michal Gregor, Katarína Zábovská, Vladimír Smataník // 2015 IEEE 19th International Conference on Intelligent Engineering Systems (INES). — 2015. — Pp. 159–164.

10. *Groza, Adrian.* Einstein Puzzles / Adrian Groza // Modelling Puzzles in First Order Logic. — Cham: Springer International Publishing, 2021. — Pp. 103–130. https://doi.org/10.1007/978-3-030-62547-4_6.

11. *Berman, Shmuel.* Solving Zebra Puzzles Using Constraint-Guided Multi-Agent Systems. — 2024. — 07.

12. cvc5: A Versatile and Industrial-Strength SMT Solver / Haniel Barbosa, Clark W. Barrett, Martin Brain et al. // Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings, Part I / Ed. by Dana Fisman, Grigore Rosu. — Vol. 13243 of *Lecture Notes in Computer Science.* — Springer, 2022. — Pp. 415–442. https://doi.org/10.1007/978-3-030-99524-9_24.

13. Review of game theory applications for situation awareness / Erik Blasch, Dan Shen, Khanh D. Pham, Genshe Chen // Sensors and Systems for Space Applications VIII / Ed. by Khanh D. Pham, Genshe Chen; International Society for Optics and Photonics. — Vol. 9469. — SPIE, 2015. — P. 94690I. https://doi.org/10.1117/12.2177531.

14. Serious games as research instruments - Do's and don'ts from a cross-case-analysis in transportation / M. Freese, H. K. Lukosch, J. Wegener, A. König // *European Journal of Transport and Infrastructure Research.* — 2020. — Vol. 20, no. 4. — Pp. 103–126.

15. Microgames in Practice: A Case Study in Container Terminal Operations / Shalini Kurapati, Daan Groen, Heide Lukosch, Alexander Verbraeck // The Shift from Teaching to Learning: Individual, Collective and Organizational Learning through Gaming Simulation. — 2014. — Pp. 333–346.

16. *Franta, W. R.* A comparison of heaps and the TL structure for the simulation event set / W. R. Franta, Kurt Maly // *Commun. ACM.* — 1978. — oct. — Vol. 21, no. 10. — P. 873–875. https://doi.org/10.1145/359619.359637.

17. *Knuth, D. E.* SOL—A Symbolic Language for General-Purpose Systems Simulation / D. E. Knuth, J. L. McNeley // *IEEE Transactions on Electronic Computers.* — 1964. — Vol. EC-13, no. 4. — Pp. 401–408.

18. *Knuth, D. E.* A Formal Definition of SOL / D. E. Knuth, J. L. McNeley // *IEEE Transactions on Electronic Computers.* — 1964. — Vol. EC-13, no. 4. — Pp. 409–414.

19. *Fishman, George S.* Discrete-event simulation / George S. Fishman. — Berlin, Heidelberg: Springer-Verlag, 2001.

Приложение А Пример диалогов в Telegram

В приложении приведены примеры диалогов, иллюстрирующие игровой цикл. Выделены *начало дня* и *конец дня*: в начале дня игрок получает текущее состояние и выбирает действие, а в конце дня подводятся итоги шага, ключевые события и обновленные значения метрик ситуационной осведомленности.

Рисунок А.1 — Лобби в групповом чате: создание игровой сессии, список игроков, таймер TimeDelay и кнопки подключения.

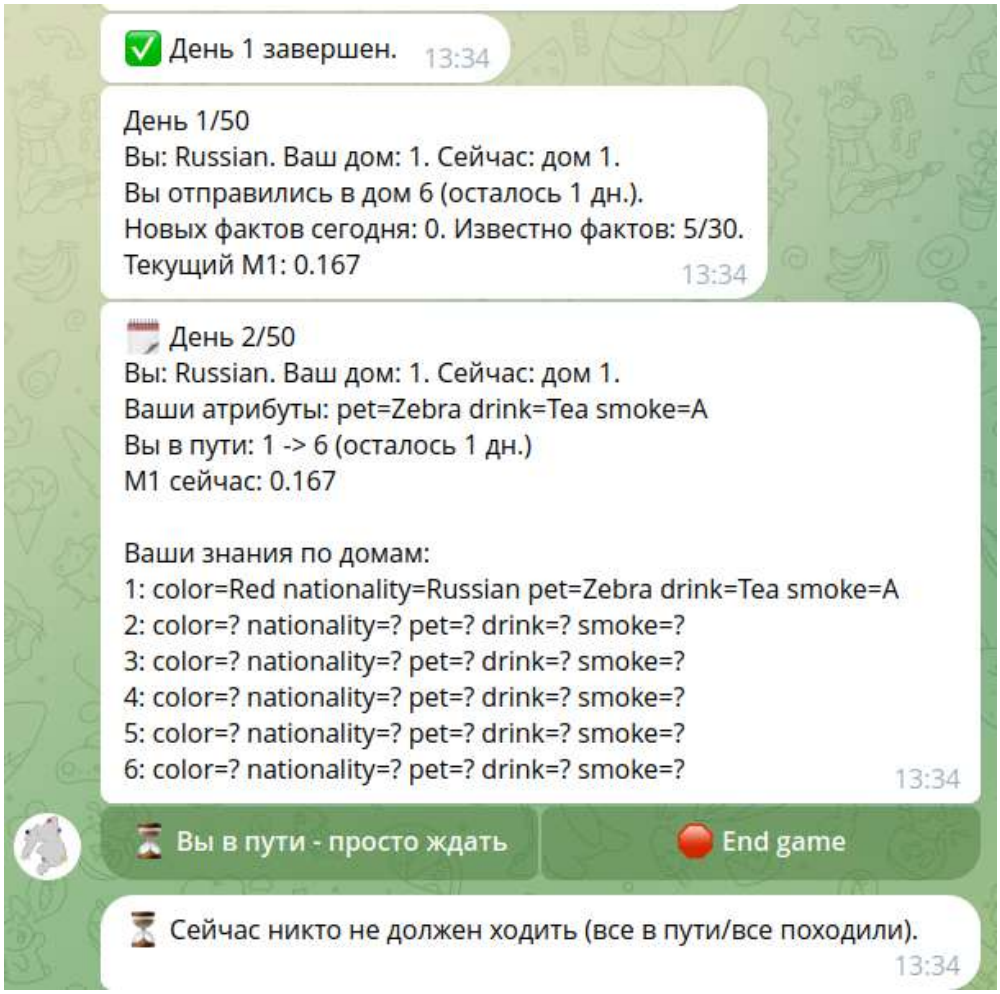

Рисунок А.2 — Начало дня (пример): игроку предоставляется полная текущая информация (позиция, атрибуты, знания по домам, значение M1) и доступные действия.

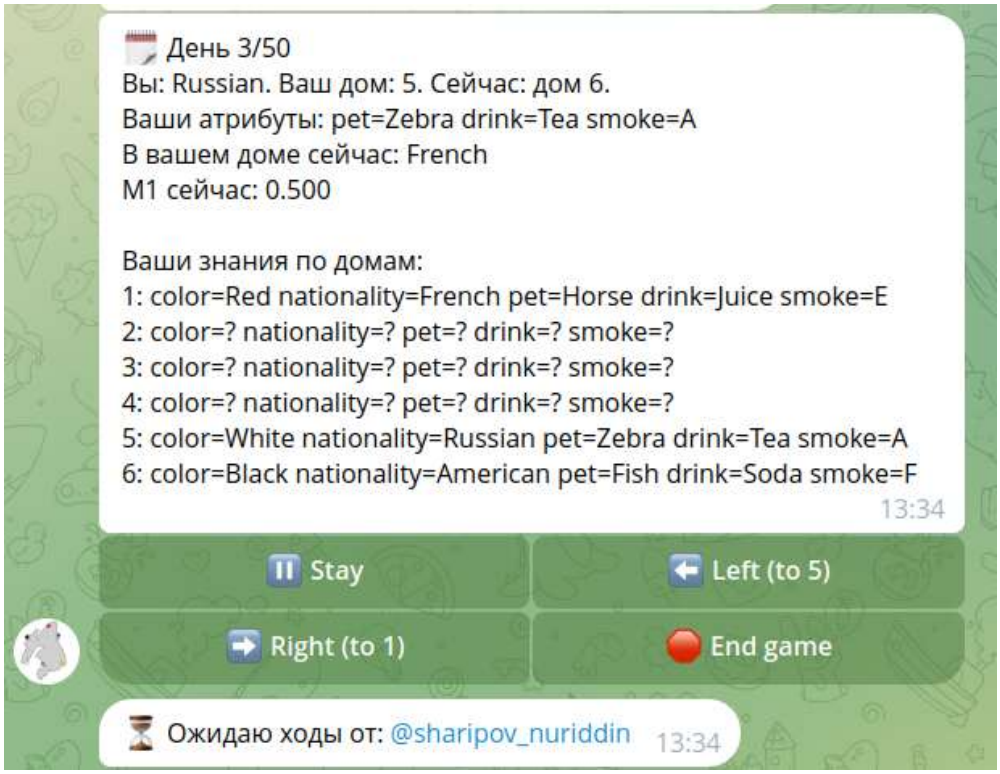

Рисунок А.3 — Выбор действия (пример): игрок принимает решение о перемещении или ожидании, что влияет на последующие наблюдения и динамику SA.

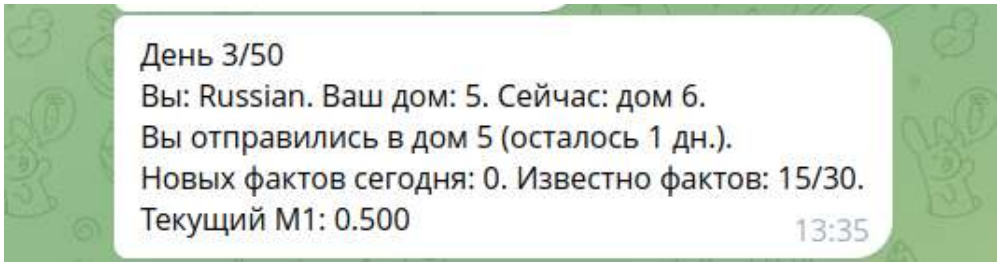

Рисунок А.4 — События дня (пример): сообщения о ходе дня и изменениях состояния (например, перемещение/прибытие и обновление текущего значения M1).

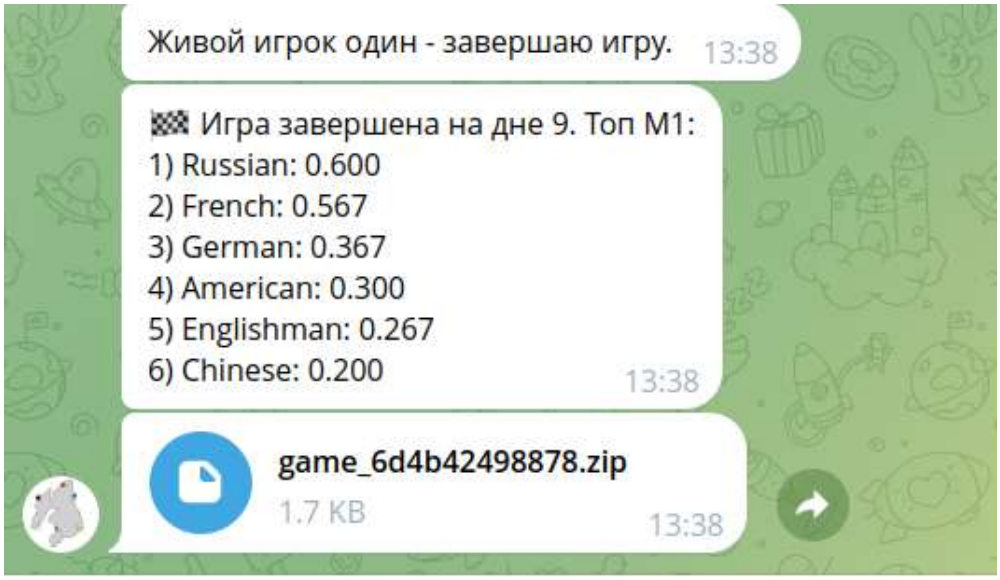

Рисунок А.5 — Конец игры (пример): завершение сессии, итоговый рейтинг по M1 и выгрузка архивом файлов логирования (CSV/XML).